SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks



Lecture 11

DevOps and Cloud Hosting Platforms

Focus: deploying the full-stack app

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@sliit.lk

Where This Lecture Fits in SE3090

Earlier	Earlier	Prev	TODAY	Goal
React frontend & .NET REST API	PostgreSQL, auth & security	Git, CI/CD & code quality	DevOps & cloud deployment	A live, monitored full-stack app

- **The capstone of the module: take everything built so far and ship it to the cloud**
- Builds directly on the CI/CD pipeline from the previous lecture — deployment is the final stage
- You will deploy the SAME stack you've been building: React + ASP.NET Core + PostgreSQL
- Directly assessed: the Main Assignment expects a deployed, working, full-stack application

Lecture Learning Outcomes

LO2

Apply suitable frameworks & tools

Deploy a full-stack application (frontend, backend API and database) to appropriate cloud hosting platforms with correct configuration.

LO3

Use best practices for delivery & operations

Apply DevOps workflows, environment configuration, secret management, monitoring, logging and maintenance to a deployed system.

LO4

Select & justify approaches

Choose and justify suitable hosting platforms and deployment strategies based on cost, scalability, security and reliability.

Why DevOps & Cloud Hosting Matter

Software has to ship

Code that isn't deployed delivers zero value. Users only ever see the running app, never your repo.

Faster, safer delivery

DevOps + automation let teams release frequently and reliably — from once a year to many times a day.

Cloud removes the heavy lifting

No buying servers or running a data centre — provision infrastructure in minutes, pay for what you use.

Scale & reliability on demand

Handle 10 users or 10 million; the platform adds capacity, backups and uptime you couldn't build alone.

Lecture Roadmap – 3 Hours

1	**DevOps culture & practices** Mindset · collaboration · automation · shared responsibility	~30 min
2	**Build, test, deploy & monitor workflows** The DevOps pipeline · deployment strategies · feedback loops	~30 min
3	**Cloud hosting platforms** IaaS/PaaS/SaaS/serverless · the landscape · choosing one	~35 min
4	**Deploying frontend & backend** Static/SPA hosting · API hosting · CORS · health checks · errors	~40 min
5	**Databases, configuration & monitoring** Managed DBs · env vars & secrets · logging · alerts · maintenance	~40 min

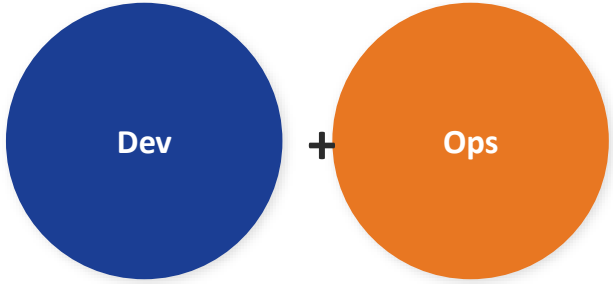
PART 1

DevOps Culture & Practices

- What DevOps is — and isn't
- Breaking down the Dev–Ops wall
- Mindset: automation, shared responsibility, continuous improvement
- DevOps vs traditional delivery, and its benefits

What Is DevOps?

- **DevOps = Development + Operations working as ONE team**
  - A culture, set of practices & tools that shorten the path from code to production
- **Goal: deliver software faster, more reliably and more frequently**
- **Built on automation, collaboration and fast feedback**
- **NOT just a tool or a single person's job title**
  - NOT only CI/CD — that's one important piece of it



DevOps

Build, ship and run software together — continuously.

Breaking Down the Dev–Ops Wall

❌ Traditional silos

- Dev writes code, 'throws it over the wall' to Ops
- Ops deploys & operates, often with little context
- Conflicting goals: features vs stability
- Slow, manual, risky releases
- Blame when things break: 'works on my machine' vs 'your code broke prod'
- Problems found late, fixed slowly

✓ DevOps collaboration

- One team owns build → ship → run
- Shared responsibility & shared tools
- Aligned goal: deliver value reliably
- Automated, frequent, low-risk releases
- Blameless culture: fix the system, not people
- Fast feedback: problems caught & fixed early

The DevOps Mindset — CALMS

Culture	Collaboration, shared ownership, blameless learning
Automation	Automate builds, tests, deployment & infrastructure
Lean	Small batches, eliminate waste, continuous improvement
Measurement	Monitor everything; decisions driven by data & feedback
Sharing	Open knowledge, tools & responsibility across the team

Automation, Improvement & Shared Responsibility

Automate everything repetitive

Builds, tests, deployments and infrastructure. Consistent, fast and error-free — humans focus on solving problems, not clicking steps.

Continuous improvement

Every release and incident is a chance to learn. Small, frequent changes + retrospectives make the system and team better over time.

Shared responsibility

'You build it, you run it.' The whole team owns quality, deployment and uptime — not a separate team to blame.

DevOps vs Traditional Delivery

Aspect	Traditional	DevOps
Teams	Separate Dev & Ops silos	Collaborative, shared ownership
Releases	Rare, large, scheduled	Frequent, small, on-demand
Deployment	Manual, error-prone	Automated pipelines (CI/CD)
Feedback	Slow; problems found late	Fast; monitored continuously
Risk	High per release	Low — small changes, easy rollback
On failure	Blame the other team	Blameless; fix the system

The shift: from rare, risky, manual, blame-heavy releases → frequent, safe, automated, collaborative delivery.

Benefits of DevOps

Faster delivery

Ship features and fixes to users in minutes/hours, not months — competitive advantage.

Higher reliability

Automation + monitoring mean fewer failures and faster recovery when they happen.

Better scalability

Automated, repeatable infrastructure scales smoothly as demand grows.

Stronger collaboration

Shared goals and tools break silos; happier, more productive teams.

Continuous feedback

Monitoring and fast loops drive constant, data-driven improvement.

Security built in

'DevSecOps' bakes security checks into the pipeline, not bolted on at the end.

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 In one sentence: what is DevOps, and why is it NOT just 'a tool' or 'one person's job'?

Q2 Before DevOps, what conflicting goals did Dev and Ops teams have?

Q3 Why is deploying many small changes often SAFER than one big release?

Q4 What does a 'blameless culture' mean, and why does it improve reliability?

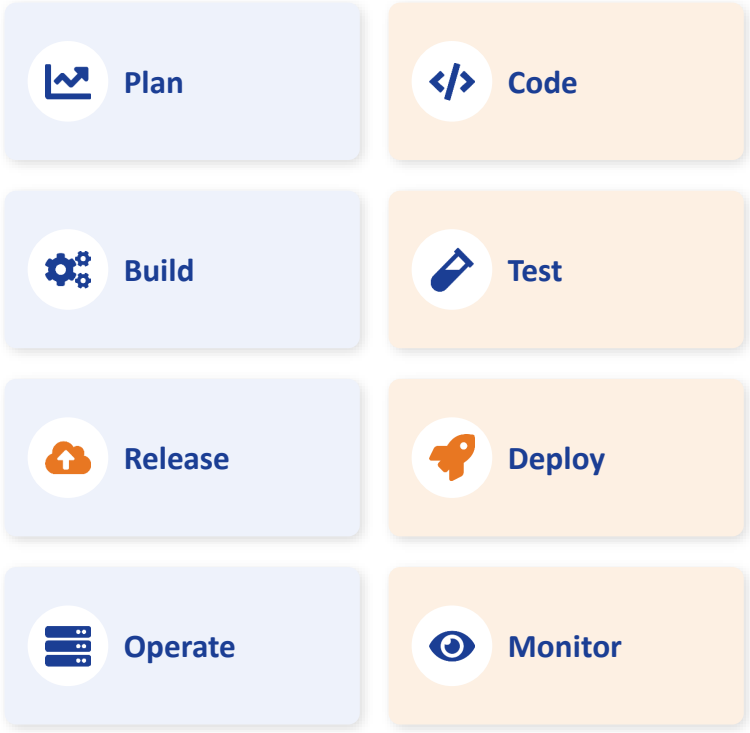
PART 2

Build, Test, Deployment & Monitoring Workflows

- The software delivery lifecycle & the DevOps loop
- Build, test, deploy and monitor workflows
- Deployment strategies: rolling, blue-green, canary
- Feedback loops & the CI/CD connection

The Software Delivery Lifecycle

- **The journey from an idea to running, monitored software**
  - DevOps makes this a continuous CYCLE, not a one-way line
- **Each stage feeds the next; monitoring feeds back to planning**
- **Automation connects the stages into a smooth pipeline**
- **The faster and safer the loop, the more value you deliver**



Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (back to Plan)

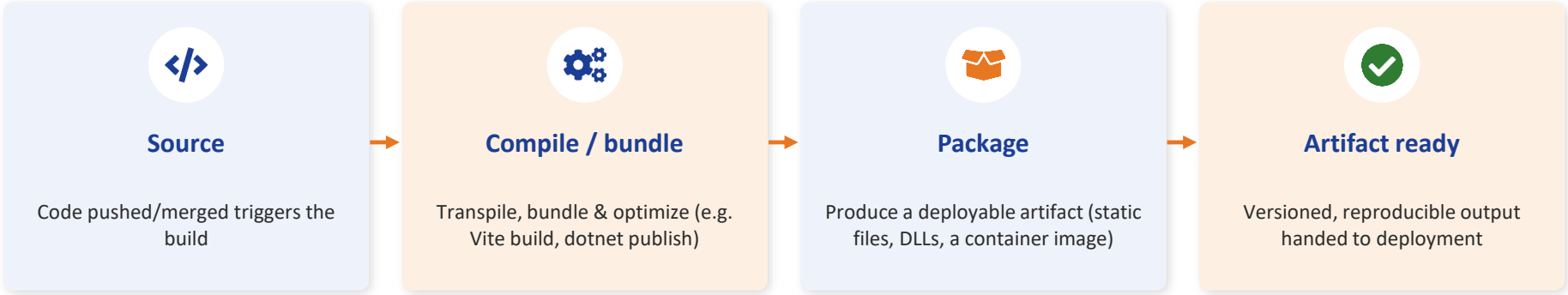
The DevOps Pipeline — A Continuous Loop



Why a loop?

- The two halves connect: CI (build & test code) flows into CD (release, deploy & run it); monitoring feeds back to planning
- It never stops — continuous integration, delivery and improvement around the loop

Build Workflow



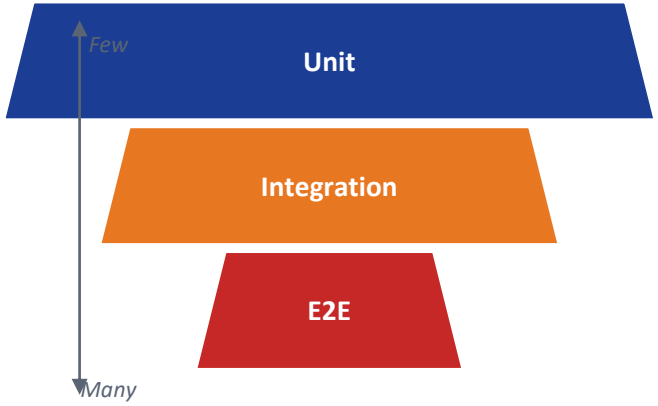
Key ideas

- **Build from a CLEAN checkout every time — reproducible, no 'works on my machine'**
- Frontend: npm run build → optimized static files in a dist/ or build/ folder
- Backend: compile to runnable output (dotnet publish) or a container image
- **The same artifact is promoted through environments — build once, deploy many times**

Automated Testing Workflow

- **Tests run automatically in the pipeline on every change**
- **A failing test STOPS the pipeline — broken code never deploys**
- **The testing pyramid:**
  - Unit tests – many, fast tests for individual functions/components.
  - Integration tests – check whether different components work together.
  - End-to-End (E2E) tests – test complete user workflows.

Gates quality before release; catches regressions instantly



Lots of fast unit tests at the base; a few slow end-to-end tests at the top.

Deployment Workflow & Strategies

Deployment means moving the tested application into an environment where users can access it.

Normally, software moves through **Development → Staging → Production**.

There are three common deployment strategies :-



Rolling Deployment

Gradually replace old instances with the new version. Usually no downtime.



Blue-Green Deployment

Maintain two environments. Blue = current version, Green = new version. Switch traffic to Green when ready.



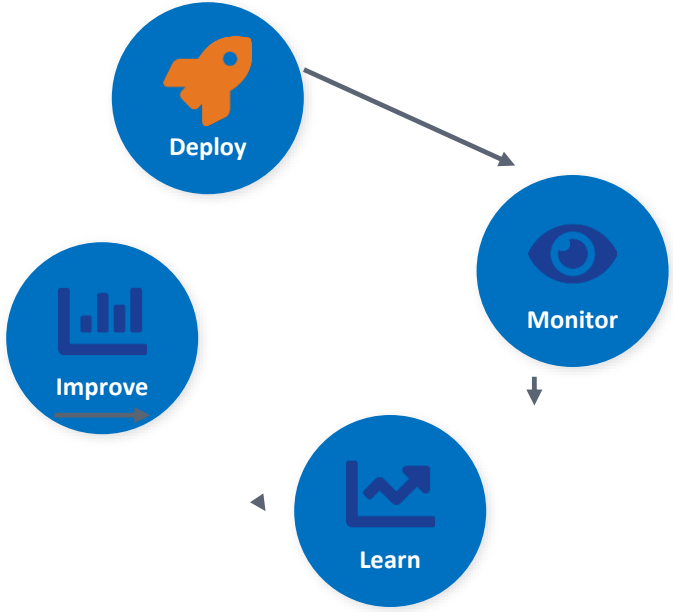
Canary Deployment

Release the new version to a small percentage of users first and monitor it before releasing to everyone.

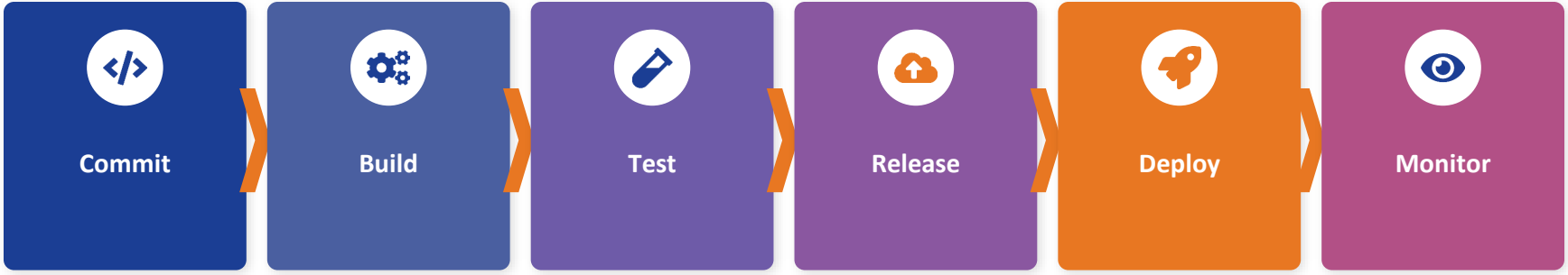
Always keep ROLLBACK easy. If a deploy goes wrong, you must be able to return to the last good version fast. Blue-green and canary make rollback near-instant; that safety net is what makes frequent deployment possible.

Monitoring Workflow & Feedback Loops

- **Deployment is the START of operations, not the end**
  - Monitor the live system: is it up? fast? error-free?
- **Collect logs, metrics, errors and uptime continuously**
- **Alerts notify the team the moment something goes wrong**
- **Feedback loop: monitoring data → informs the next changes**
  - Closes the DevOps loop back to planning & improvement



How CI/CD Connects It All



CI — Continuous Integration

- Commit → Build → Test, **automatically**
- Catches integration bugs early
- Produces a tested artifact

CD — Continuous Delivery/Deployment

- Release → Deploy the tested artifact
- Delivery = one click; Deployment = automatic
- Then operate & monitor in production

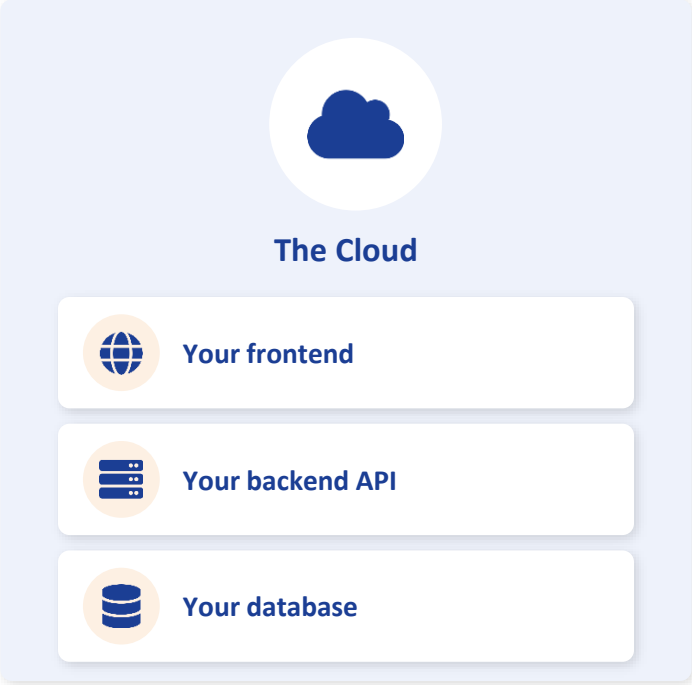
PART 3

Cloud Hosting Platforms

- What cloud hosting is & why teams use it
- IaaS, PaaS, SaaS & serverless
- The platform landscape: AWS, Azure, GCP, Vercel, Netlify, Render…
- Choosing a platform: cost, scale, security, reliability

Introduction to Cloud Hosting

- **Cloud hosting = running your app on someone else's managed servers, over the internet**
  - Rent computing power instead of owning physical machines
- **Provision resources in minutes, on demand**
- **Pay-as-you-go: pay only for what you use**
- **Provider handles hardware, networking, power, cooling & much of the maintenance**
  - Accessible from anywhere; scales up or down as needed



Why Modern Systems Use Cloud Platforms

Lower upfront cost

No buying servers; pay-as-you-go turns big capital costs into small running costs. Free tiers for small projects.

Elastic scalability

Scale up for a traffic spike, down when quiet — automatically. Handle growth without re-architecting.

Reliability & uptime

Redundancy, backups and SLAs you couldn't build yourself; data centres worldwide.

Speed & ease of deployment

Go from code to live in minutes; managed services remove most ops work.

Global reach

Serve users worldwide with low latency via regions & CDNs.

Built-in security

Providers invest heavily in security, compliance & DDoS protection.

Cloud Service Models



IaaS

Infrastructure as a Service

we rent infrastructure such as virtual machines, but we manage the **OS, runtime, and application.** Max control & work.

AWS EC2, Azure VMs



PaaS

Platform as a Service

we mainly provide our **application code**, while the platform manages the servers, OS, and runtime.

Render, Railway, Heroku, App Service



Serverless

Functions / managed

Run code per-request; auto-scales to zero. No servers to think about at all.

AWS Lambda, Cloud Functions, Vercel



SaaS

Software as a Service

Ready-made software you just use — no deployment. The end-user model.

Gmail, Figma, Notion

More control & responsibility ← IaaS

SaaS → less control, less work

Who Manages What? (Responsibility)

	On-Prem	IaaS	PaaS	SaaS
Application	You	You	You	Provider
Data	You	You	You	Provider
Runtime	You	You	Provider	Provider
OS	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers	You	Provider	Provider	Provider
Networking	You	Provider	Provider	Provider

The Cloud Platform Landscape

The hyperscalers — full-service, everything-as-a-service (steeper learning curve)

- **AWS**
- **Microsoft Azure**
- **Google Cloud (GCP)**

Developer-friendly platforms — simple, fast, great free tiers (ideal for your project)

Vercel Frontend / Next.js	**Netlify** Frontend / static	**Render** Full-stack / APIs	**Railway** Full-stack / APIs
Heroku Full-stack (classic)	**Firebase** Google BaaS	**Supabase** Postgres + backend	**Neon** Serverless Postgres

Cloud Platform Comparison

Platform	Best for	Model	Free tier?
AWS / Azure / GCP	Large-scale, full control, enterprise	IaaS + PaaS + serverless	Limited / credits
Vercel	Frontend & Next.js apps	Serverless / PaaS	Yes — generous
Netlify	Static sites & SPAs	Serverless / PaaS	Yes — generous
Render / Railway	Full-stack apps & APIs	PaaS	Yes
Heroku	Quick full-stack (classic)	PaaS	No (paid)
Supabase / Neon	PostgreSQL + backend services	Managed DB / BaaS	Yes

No single 'best' platform — it depends on the project. Match the tool to your requirements & justify it (LO4).

Selecting a Hosting Platform (LO4)

Cost

Free tier limits, pricing model, cost at scale. Predictable for a student project?

Scalability

Can it grow with demand? Auto-scaling? Will you hit limits soon?

Security

HTTPS by default, access controls, compliance, DDoS protection.

Reliability

Uptime/SLA, redundancy, backups, track record.

Ease of deployment

Git integration, build automation, learning curve, good docs.

Ecosystem fit

Supports your stack (.NET, Node, Postgres)? Integrations you need?

Checkpoint 02 – Discussion

Discuss in pairs — 5 minutes, then we compare answers

D1 Explain IaaS vs PaaS vs serverless to a friend — using the 'how much do YOU manage?' idea.

D2 On PaaS, name two things the provider handles and two things you're still responsible for (incl. security).

D3 Why might a student team pick Vercel + Render + Supabase over raw AWS for their project?

D4 Your client needs strict data-residency compliance and 99.99% uptime. Does that change your platform choice? How?

PART 4

Deploying the Frontend & Backend

- Frontend: build output, static/SPA hosting, domains, HTTPS & CDN
- Backend: API hosting, server vs container, CORS & health checks
- Build commands & output folders
- Common deployment errors & how to fix them

Frontend Build & Static / SPA Hosting

- **A React app isn't served as-is — it's BUILT into static files first**
  - npm run build → optimized, minified HTML, CSS & JS bundles
- **Output is a folder of plain static assets (dist/ or build/)**
- **Static hosting + a global CDN serve those files fast, worldwide**
- **An SPA is ONE index.html that swaps views with JavaScript**
  - So the host must fall back to index.html for every route

build output

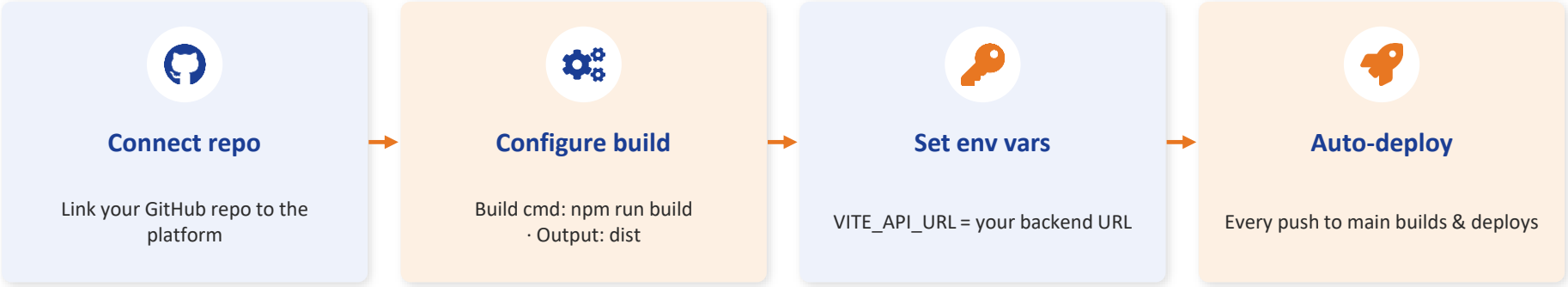
```
# build the frontend
npm run build

dist/              # the deployable output
  ├─ index.html
  ├─ assets/app-3f9.js    # hashed
  └─ assets/app-3f9.css
```

SPA routing gotcha

Refreshing /products asks the server for a file that doesn't exist → 404. Fix: configure a rewrite so ALL paths serve index.html and React Router handles them.

Frontend Deployment Workflow (Vercel / Netlify)



Typical platform settings

Setting	Value
Framework preset	Vite / Create React App
Build command	npm run build
Output directory	dist (or build)
Install command	npm ci

Why these platforms?

- Zero-config for popular frameworks
- Automatic HTTPS + global CDN
- Git-based: push to deploy
- Free tier ideal for students
- Preview deploys per PR

Domains, HTTPS, CDN & Common Frontend Errors

Going live properly

- **Custom Domain**: connect your own domain to the hosting platform using DNS records such as CNAME or A records.
- **HTTPS**: modern hosting platforms can provide TLS certificates automatically, allowing secure HTTPS access.
- **CDN**: static files are cached on edge servers closer to users, improving loading speed.
- **Cache-busting:** hashed filenames help ensure users receive the latest version of files after deployment.

Common frontend deploy errors

- **Blank page — wrong output dir or base path**
- **404 on refresh — missing SPA index.html fallback**
- **API calls fail — wrong/missing API URL env var or CORS**
- **Assets 404 — incorrect base / sub-path config**
- **Old version showing — stale cache; redeploy / hard refresh**

Backend API Hosting Concepts

- **A backend is a PROCESS that must keep running to serve requests**
  - Unlike static frontends, it needs CPU, memory & a runtime alive 24/7
- **Two common hosting models:**



Server-based (PaaS)

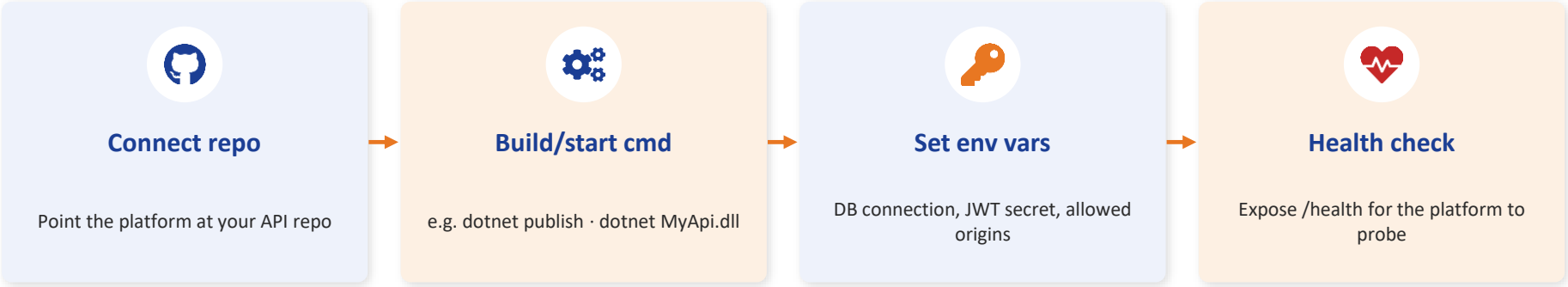
- We push our code, and the **platform builds and runs the application.**
- The platform handles areas such as the **OS, runtime, scaling, and infrastructure management.**
- *Examples include* Render, Railway, Azure App Service, and Heroku.
- This is generally a simpler approach because developers can focus mainly on the application.



Container-based (Docker)

- We package the **application together with its dependencies** into a container image.
- This helps the application run consistently across different environments — reducing the “works on my machine” problem.
- Examples of platforms/services that can run containers include Google Cloud Run, AWS ECS, and Kubernetes.
- Containers provide more portability and control, but they also introduce additional concepts to learn.

Backend API Deployment Workflow



Where to host a REST API

Render PaaS, simple	**Railway** PaaS, fast setup	**Azure App Service** Enterprise PaaS
AWS Beanstalk Managed AWS	**Cloud Run** Containers	**Heroku** Classic PaaS

Base URLs, CORS & Health Checks

```
// FRONTEND: call the deployed API, not localhost
const API = import.meta.env.VITE_API_URL;
fetch(`${API}/api/products`)

// BACKEND (ASP.NET Core): allow the frontend origin
app.UseCors(p => p
  .WithOrigins("https://lankamart.vercel.app")
  .AllowAnyHeader().AllowAnyMethod());

// health check endpoint
app.MapGet("/health", () => Results.Ok("OK"));
```

The 3 things that break

- **Base URL: frontend must point at the DEPLOYED API, never localhost**
- **CORS: the API must explicitly allow the frontend's origin**
- **Health: expose /health so the platform knows you're alive**

CORS errors are fixed on the BACKEND (allow the origin), not the frontend — a browser security rule, not a bug in your fetch call.

Common Backend Deployment Issues

App crashes on start	Missing env var (DB URL, secret) → set them BEFORE deploy; read the logs
Wrong port binding	Bind to the platform's $PORT, not a hardcoded 5000
Can't reach database	Wrong connection string, or DB firewall blocks the host
CORS blocked	Add the frontend origin to the API's CORS allow-list
Secrets exposed / missing	Use platform env vars; never commit .env or hardcode keys
Cold starts (free tier)	Idle apps sleep; first request is slow — expected on free plans

When in doubt: READ THE LOGS — the platform's log output names the exact failure almost every time.

Checkpoint 03 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 Why does a React app need a BUILD step before deployment, and what is the output?

Q2 A deployed SPA returns 404 when you refresh on /products. What's the cause and fix?

Q3 The deployed frontend can't call the deployed API (CORS error). Which side do you fix?

Q4 Your API crashes on startup in the cloud. What's the most likely cause and your first step?

PART 5

Databases, Configuration, Monitoring & Maintenance

- Cloud database hosting & managed PostgreSQL
- Environment variables & production configuration
- Monitoring, logging & error tracking
- Uptime, alerts, maintenance & continuous improvement

Cloud Database Hosting & Managed PostgreSQL

- **Don't run your own database server in production — use a MANAGED one**
  - The provider handles backups, patching, scaling & availability
- **You get a connection string; you focus on your data**
- **Managed PostgreSQL options:**
  - Supabase, Neon — modern, generous free tiers (great for students)
  - Render PostgreSQL, Railway PostgreSQL — sit beside your app
  - AWS RDS, Azure Database for PostgreSQL — enterprise-grade

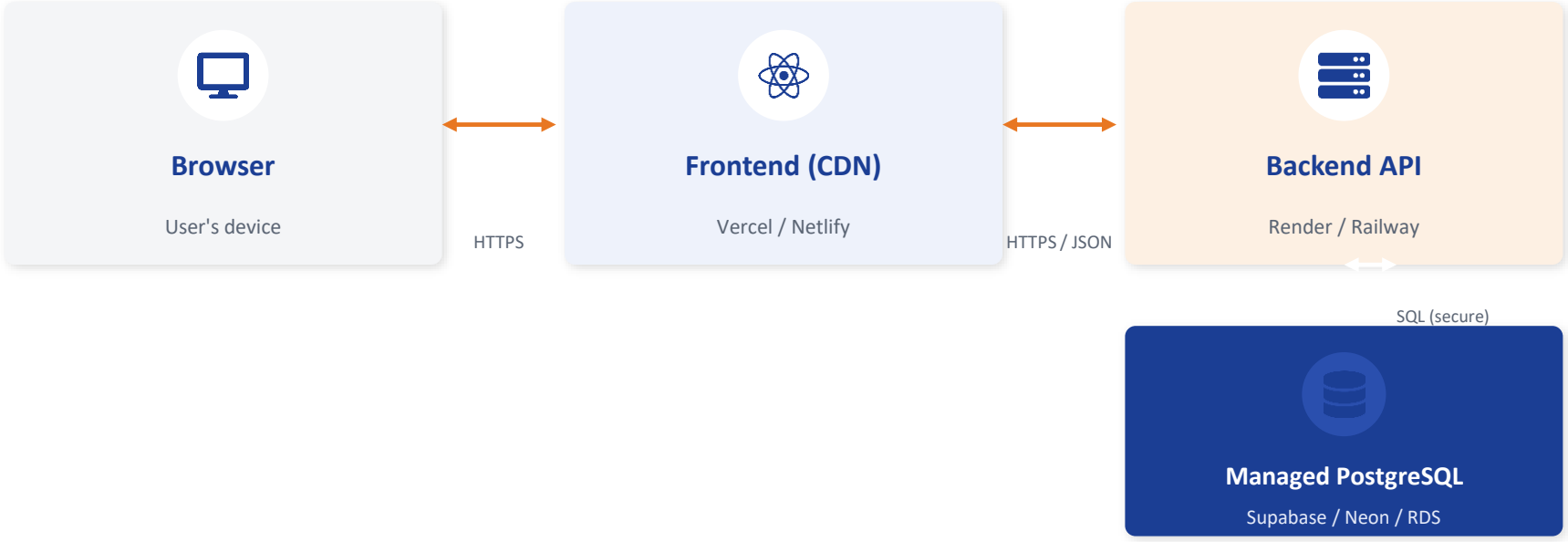
Managed handles for you

- Automated backups & restore
- Security patches & updates
- Scaling & failover
- Monitoring & metrics

You still own

- Schema design & migrations
- Keeping the connection string secret
- Sensible access rules
- Your data's correctness

Full-Stack Hosting Architecture



Each tier is hosted separately, connected over HTTPS — and only the backend talks to the database.

Connection Strings, Backups & DB Security

```
connection string

# a PostgreSQL connection string (a SECRET!)
postgresql://user:password@host:5432/dbname

# stored as an env var, never in code
DATABASE_URL=postgresql://...  # platform setting
```

- **Treat the connection string as a top secret credential**
  - Inject via env vars; rotate if ever exposed
- **Backups: managed providers back up automatically — know how to RESTORE**
  - Test restores; a backup you can't restore isn't a backup

Hosted database security checklist

- ✓ Strong, unique credentials — never the defaults
- ✓ Require SSL/TLS for connections
- ✓ Restrict access: only the backend's host/IP, not the public internet
- ✓ Least privilege: the app's DB user gets only what it needs
- ✓ Encrypt data at rest (managed providers do this)
- ✓ Regular backups + a tested restore plan

Environment Variables & Production Config

- **Env vars = configuration that lives OUTSIDE your code**
  - Same code, different values per environment (dev / staging / prod)
- **Hold secrets & settings: DB URL, JWT secret, API keys, base URLs**
- **Never hardcode these — keep them out of the repo entirely**
- **Locally: a git-ignored .env file. In production: platform settings**

```
.env  (local only, git-ignored)
DATABASE_URL=postgresql://...
JWT_SECRET=super-long-random-value
VITE_API_URL=https://api.lankamart.app

# read in code (never the literal!)
process.env.JWT_SECRET   // backend
import.meta.env.VITE_API_URL // frontend
```

Frontend vs backend env vars — a critical difference

- **Backend env vars are PRIVATE (on the server) → safe for real secrets like JWT_SECRET & DB passwords**
- **Frontend (VITE_*) vars are BUNDLED INTO PUBLIC JS → visible to anyone; only put non-secret values there**

Monitoring: Why It Matters After Deployment

- **Deployment isn't the finish line — it's the start of operations**
  - Without monitoring you learn about outages from angry users
- **Goal: detect & diagnose problems BEFORE (or faster than) users report them**



Uptime

Is the app reachable & responding? Track availability (e.g. 99.9%).



Performance

How fast? Response times, latency, throughput, slow endpoints.



Errors

What's failing? Error rates, exceptions, failed requests.



Logs

What happened & why? The detailed record for diagnosis.

Logging & Error Tracking

Good logging

- **Logging** records what happens while the application is running.
- Use log levels: **DEBUG, INFO, WARN, ERROR**.
- Log meaningful information with context such as **timestamp, request ID and user ID.**
- Never log **passwords, tokens, or other secrets.**
- **Structured logs**, such as JSON logs, are easier to search and filter.

Error tracking

- **Error-tracking tools** such as Sentry automatically capture application exceptions.
- They provide **stack traces, error context, affected requests/users, and alerts**.
- Duplicate errors can be grouped together, making problems easier to investigate.

Uptime Monitoring, Alerts & Incident Response

Status dashboard

API	Operational	120 ms
Frontend	Operational	45 ms
Database	Degraded	high load
Auth	Operational	98 ms

- **Uptime monitors ping your app from outside, 24/7**
- **Down? You get alerted (email / SMS / Slack) in seconds**
- **Set alerts on thresholds: errors, latency, downtime**
  - Avoid alert fatigue — alert on what's actionable

Incident response (when something breaks)

Detect (alert fires) → Acknowledge → Diagnose (logs/metrics) → Mitigate / rollback → Fix & verify → Post-mortem

Maintenance & Continuous Improvement

Update dependencies

Keep libraries current for fixes & features; review before merging (Dependabot).

Apply security patches

Patch known vulnerabilities promptly — in your code, deps & runtime.

Tend the database

Monitor growth, add indexes, archive old data, verify backups.

Improve continuously

Use monitoring data to find slow spots & errors; iterate and optimise.

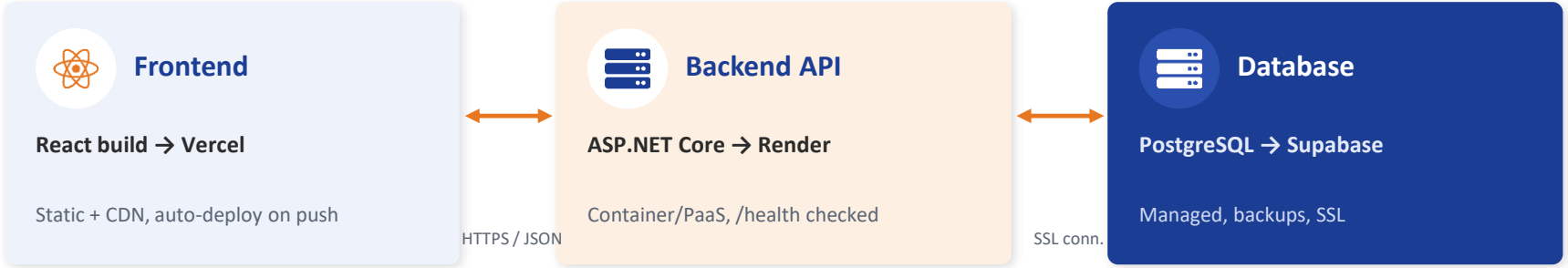
PART 6

Case Study, Activities & Wrap-Up

- End-to-end: deploying a full-stack app
- Group task: choose a hosting stack
- Deployment troubleshooting challenge
- Quiz, summary & what's next

Case Study: Deploying Full-Stack LankaMart

One commit → three hosted services, wired together by configuration



The configuration that wires them together

```
# Vercel (frontend) env
VITE_API_URL=https://lankamart-api.onrender.com
# Render (backend) env
DATABASE_URL=postgresql://...supabase.co:5432/db
JWT_SECRET=••••••   ALLOWED_ORIGINS=https://lankamart.vercel.app
```

Case Study: The Deployment Decisions

How the team deploys safely (practice → why — LO4)

Practice	What the team does	Why
Platform choice	PaaS (Vercel / Render / Supabase), not raw VMs	Small team, fast setup, managed ops — low overhead
Auto-deploy	Push to main → CI/CD builds, tests & deploys	Fast, repeatable releases; no manual FTP
Config	All URLs, keys & secrets in env variables	Same code in dev & prod; no secrets in the repo
CORS & health	Backend allow-lists the frontend; /health endpoint	Browser calls succeed; platform restarts on failure
Database	Managed PostgreSQL with backups + SSL	Durable, secure data without DBA effort
Monitoring	Uptime check + error tracking + alerts	Know about problems before users complain

Group Activity: Choose a Hosting Stack (20 min)

Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: "EventLanka" ticketing app

React frontend + ASP.NET Core REST API + PostgreSQL. Small team, low budget, spiky traffic around big event launches.

- **Task 1 — Pick a host for the frontend, backend & database; name each**
- **Task 2 — IaaS, PaaS or serverless for the API? Justify it**
- **Task 3 — List the environment variables each service needs**
- **Task 4 — Which selection criteria drove your choices (cost, scale, ease…)?**

Marking lens (= LO4)

- Sensible managed platforms for a small team?
- Choice JUSTIFIED against criteria, not just named?
- Service model (PaaS/serverless) fits spiky traffic?
- Env vars cover API URL, DB URL, JWT secret, CORS?
- Trade-offs acknowledged (cost vs control vs scale)?

Activity: Diagnose the Deployment

Match each symptom to its likely cause & fix — 6 min in pairs

Symptom	Likely cause	Fix
Frontend loads; all API calls fail with CORS error	Backend doesn't allow the frontend origin	Add the frontend URL to ALLOWED_ORIGINS / CORS policy
Frontend works locally, blank/404 on refresh in prod	SPA routing not configured on the host	Add SPA fallback rewrite to index.html
Backend crashes on startup in the cloud	Missing env var (e.g. DATABASE_URL)	Set the env vars in the platform; read the logs
App can't reach the database	Wrong connection string / SSL / IP not allowed	Fix DATABASE_URL; enable SSL; allow-list / use pooled URL
Calls go to localhost in production	API base URL hardcoded, not from env	Use VITE_API_URL; rebuild & redeploy
First request after idle is very slow	Free-tier cold start (instance slept)	Expected on free tier; use paid/always-on or warmups

Golden rule of deployment debugging: READ THE LOGS first — they almost always name the real problem.

Quick Quiz — 6 Questions

1	DevOps is BEST described as…	A) a tool B) a culture of dev+ops collaboration C) a cloud
2	You manage code only; the platform runs servers & scaling. This is…	A) IaaS B) PaaS C) on-prem
3	Where should a JWT secret live in production?	A) in the code B) an env variable / secret store C) the README
4	Every API call from the deployed site fails with CORS. Fix on…	A) frontend host B) backend ALLOWED_ORIGINS C) the database
5	Which gives a managed PostgreSQL with backups?	A) Vercel B) Supabase / Neon / RDS C) ESLint
6	First request after idle is slow on a free tier because of…	A) a virus B) cold start C) too many env vars

Lecture Summary — Key Takeaways

DevOps is culture + automation

Dev & ops share responsibility; automate build, test, deploy & monitor; improve continuously.

Full-stack = 3 hosted services + config

Frontend, backend & database, wired by env vars (API URL, DB URL, secrets, CORS).

CI/CD ships safely & often

A pipeline takes one commit through to a live, repeatable deployment.

Config & secrets via env variables

Never hardcode; same code in dev & prod; managed DBs for backups & security.

Cloud models trade control for ease

IaaS → PaaS → serverless: less to manage as you move up. Pick per requirement.

Deploying isn't the finish line

Monitor, log, alert, patch & maintain — operations is continuous.

Learning Outcomes, Next Steps & Preparation

LO	Covered today by…
LO2	Applying cloud platforms & tools to deploy a full-stack app: frontend, backend API, managed database & environment configuration
LO3	Best practices for delivery & operations: CI/CD workflows, secure config, health checks, monitoring, logging & maintenance
LO4	Selecting & justifying a hosting stack and service model (IaaS/PaaS/serverless) for a project (case study + activities)



Next & beyond

Wrapping the module: integrate everything into the team full-stack project — built, collaborated on with Git/CI-CD, and now deployed & operated in the cloud.



Before the lab / project

- Create free accounts: Vercel/Netlify, Render/Railway, Supabase/Neon
- Deploy a managed PostgreSQL; copy its connection string
- Deploy your API; set env vars; verify /health responds
- Deploy your frontend with VITE_API_URL; fix CORS end-to-end

References & Further Reading

- **Platform documentation (primary sources)**
  - Vercel — vercel.com/docs · Netlify — docs.netlify.com · Firebase Hosting — firebase.google.com/docs/hosting
  - Render — render.com/docs · Railway — docs.railway.app · Heroku Dev Center — devcenter.heroku.com
  - AWS — docs.aws.amazon.com · Microsoft Azure — learn.microsoft.com/azure · Google Cloud — cloud.google.com/docs
  - Databases: Supabase — supabase.com/docs · Neon — neon.tech/docs
- **DevOps & configuration concepts**
  - The Twelve-Factor App — 12factor.net (config, env vars, deployment)
  - Docker documentation — docs.docker.com · DORA / Accelerate metrics — dora.dev
- **Recommended books**
  - Kim, Behr & Spafford (2018) The Phoenix Project. IT Revolution.
  - Kim et al. (2021) The DevOps Handbook, 2nd edn. · Forsgren, Humble & Kim (2018) Accelerate.